Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Shravanthi R **1BM24EC193**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

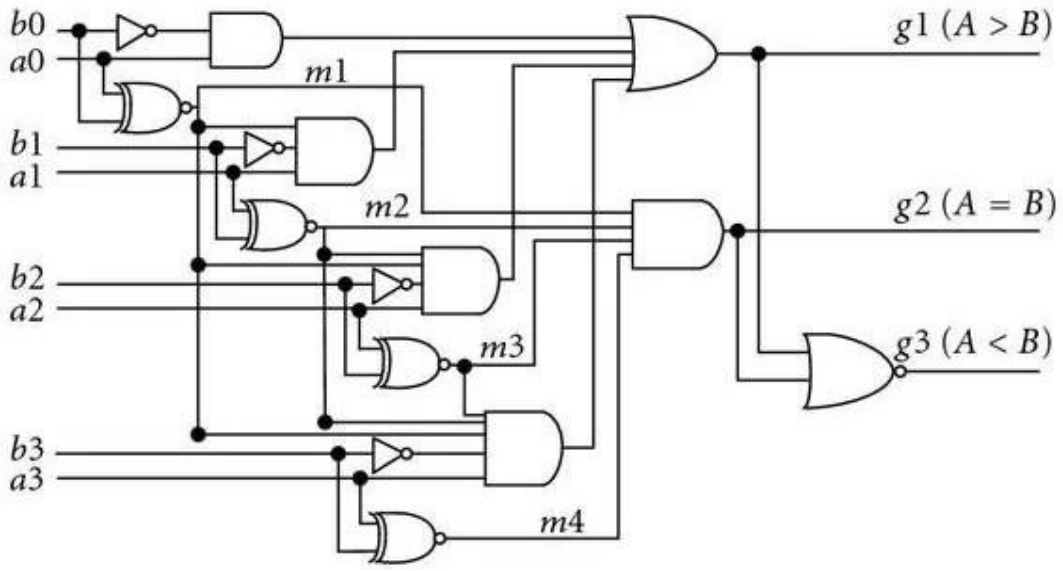
2026 – 2027

Table of Contents

Sl no.	Question	Pg no.
1	Comparator	1
2	Counter	4

Comparator

Circuit Diagram:



Design Code:

```
module comparator_4bit (
   input [3:0] A,
   input [3:0] B,
   output A_gt_B,
   output A_lt_B,
   output A_eq_B
);
assign A_gt_B = (A > B);
assign A_lt_B = (A < B);
assign A_eq_B = (A == B);

endmodule
```

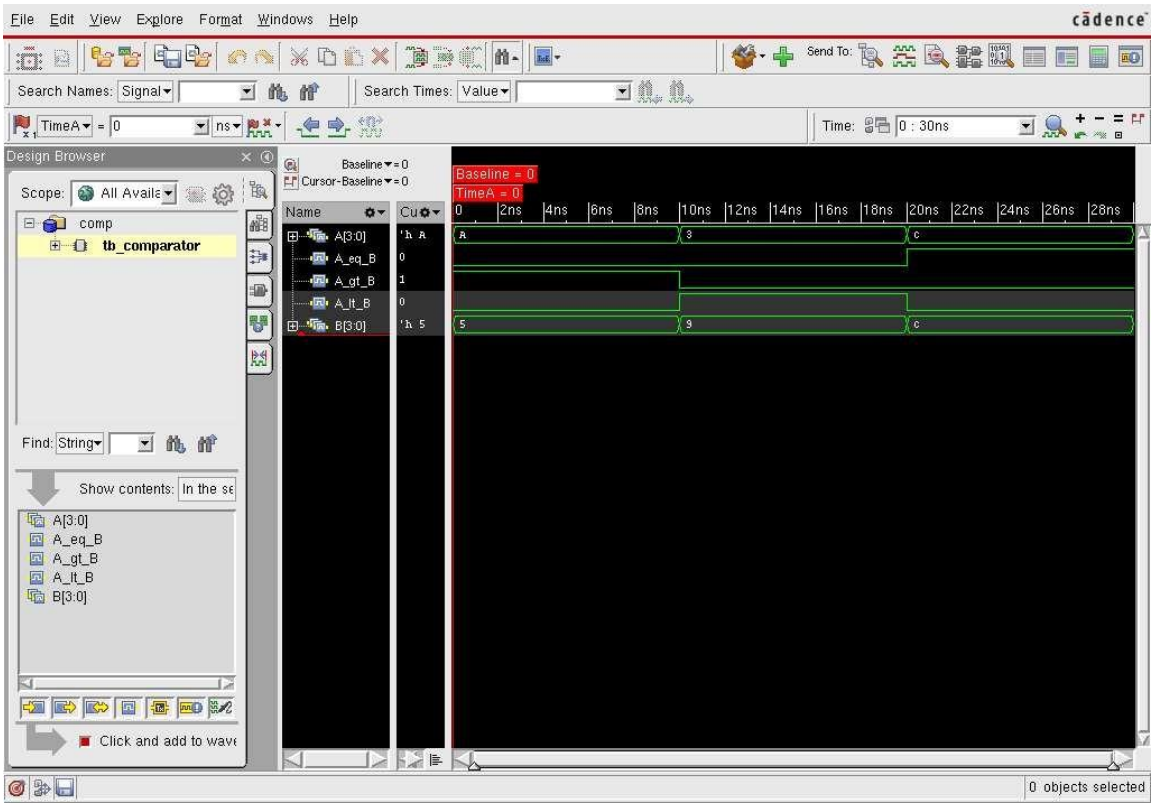
Testbench:

```
module tb_comparator;
reg [3:0] A, B;
wire A_gt_B, A_lt_B, A_eq_B;

comparator_4bit uut (
   .A(A),
   .B(B),
   .A_gt_B(A_gt_B),
   .A_lt_B(A_lt_B),
   .A_eq_B(A_eq_B)
);

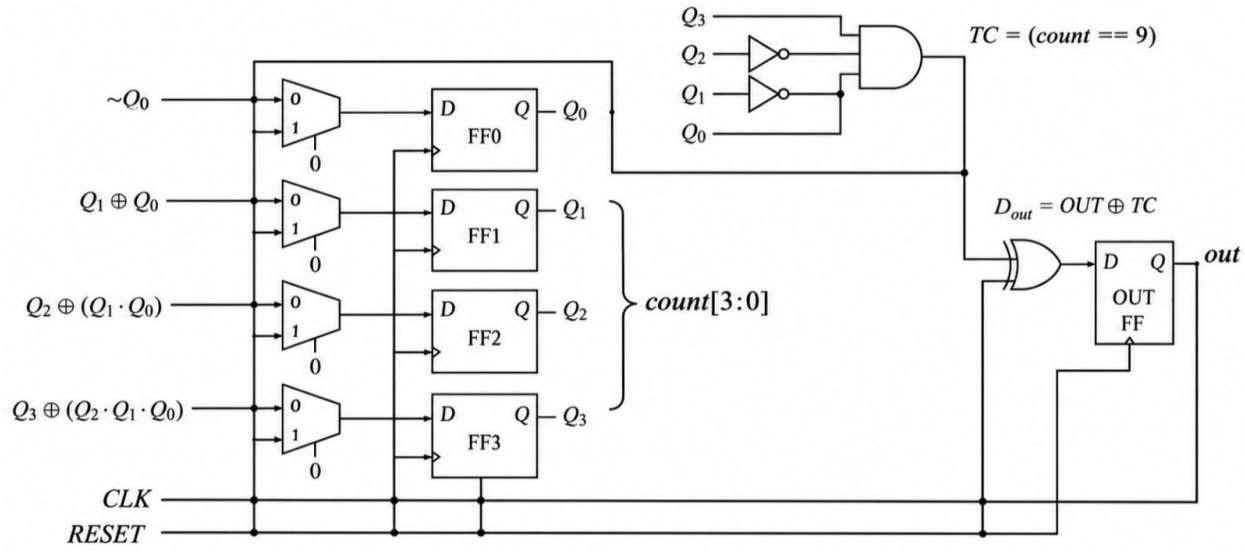
initial begin
   A = 4'b1010; B = 4'b0101;
   #10;
   A = 4'b0011; B = 4'b1001;
   #10;
   A = 4'b1100; B = 4'b1100;
   #10;
   $finish;
end
initial begin
$dumpfile("comp.vcd");
$dumpvars;
end
endmodule
```

Simulation:


File Edit View Explore Format Windows Help
cadence
Search Names: Signal
Search Times: Value
Send To:
Time: 0 : 30ns
Design Browser
Scope: All Availa
comp
tb_comparator
Find: String
Show contents: In the se
A[3:0]
A_eq_B
A_gt_B
A_lt_B
B[3:0]
Click and add to wave
Baseline = 0
Cursor-Baseline = 0
Name
Cu
TimeA = 0
0 2ns 4ns 6ns 8ns 10ns 12ns 14ns 16ns 18ns 20ns 22ns 24ns 26ns 28ns
'h A
'h 5
0 objects selected

Counter

Circuit Diagram:



Design Code:

```
module divide_n_counter #(parameter N = 10)
(
   input clk,
   input rst,
   output reg [$clog2(N)-1:0] count,
   output reg out
);

always @(posedge clk or posedge rst)

begin

   if (rst)
```

```
  begin
    count <= 0;
    out <= 0;
  end

  else
  begin
    if (count == N-1)
    begin
      count <= 0;
      out <= ~out;
    end

    else
    begin
      count <= count + 1;
    end
  end
end

endmodule
```

Testbench:

```
`timescale 1ns/1ps

module tb_counter;
reg clk, rst;
wire [3:0] count;
wire out;

divide_n_counter #(.N(10)) uut (
```

```
    .clk(clk),
    .rst(rst),
    .count(count),
    .out(out)
);
always #5 clk = ~clk;

initial
begin
    $shm_open("waves.shm");
    $shm_probe(tb_counter, "AS");
    clk = 0;
    rst = 1;
    #10 rst = 0;

    #200 $finish;
end
endmodule
```

Simulation:

